ROCKETRIDE EXPLAINED / 01

Start here: what is Redis?

A plain-language guide to your RocketRide project and the meeting. October 29, 2026.

Redis is software that an application uses to save information and retrieve it quickly. It is a kind of database: an organized place where software stores data.

Redis keeps information mainly in memory, the computer's fast working area. Depending on its setup, it can save information to disk too. For this project, focus first on saving something and getting it back.

Picture a notebook with labeled entries

Each entry has a **key**, which is its label, and a **value**, which is the information saved under that label.

Key: the label	Value: the saved information
answer_to_question_1	Redis stores information for fast access.
questions_asked_by_user_123	4

An application gives Redis the label when it wants to save or retrieve that entry. Redis does not have to understand what the saved sentence means.

The basic things Redis can do

Save: "Keep this answer under this label."
Read: "Give me the information under that label."
Delete: "Remove that entry."
Expire: "Remove this automatically after five minutes."

> You may hear **SET** for saving, **GET** for reading, and **TTL** for the time until an entry expires. These are names for the same simple actions above.

The next page shows why saving a piece of information can be useful in an AI app. You do not need to read the code before understanding that example.

ROCKETRIDE EXPLAINED / 02

Why would an AI app use Redis?

Follow one question through the app. This is the idea behind RocketRide's proposed Answer Cache app.

The first time someone asks

1. A person types: "What is Redis?"

2. The app checks Redis for a saved answer. There is no entry yet.

3. The app asks an AI model to write an answer. Generating that answer takes time and may cost money.

4. The app returns the answer and saves a copy in Redis, with an instruction to expire it after a chosen time.

The next time the same question arrives

5. The app checks Redis again. This time it finds the saved answer.

6. The app returns that answer instead of generating it again.

Caching means saving a result so you can reuse it. Redis holds the saved result. The AI model writes the answer.

Word you may hear	What it means in this example
Cache hit	The saved answer was found and can be reused.
Cache miss	No usable saved answer was found. The app needs a new one.
Expiration or TTL	The saved answer has a time limit. When it expires, the next request needs a fresh answer.

Five minutes is only an example, not an agreed project requirement. Answers that change often could have a shorter lifetime than answers about stable facts.

The app must decide when two requests are equivalent. Redis itself looks up a key; it does not automatically recognize that two differently worded questions mean the same thing. We will return to this on page 5.

ROCKETRIDE EXPLAINED / 03

What are we building for RocketRide?

There are two main technical pieces: the Redis connector and an app that demonstrates it.

RocketRide lets developers connect reusable components to build an AI workflow. A workflow is a sequence of actions, such as receiving a question, checking saved information, generating an answer, and returning it.

RocketRide calls these components **nodes**. It calls the connected workflow a **pipeline**. Here, a node simply means a component that performs a job.

Piece	Its job in our example
The app	The screen where a person asks a question and reads the answer.
The Redis node	The component that lets RocketRide send save, read, and other agreed requests to Redis.
Redis itself	The separate software that holds the saved information.

Your team's first piece: the Redis node

You write the connection between RocketRide and Redis. For example, RocketRide gives your component a label and an answer. Your component saves that answer in Redis and reports whether it worked.

For a read request, your component receives a label and returns the saved answer, or reports that no entry exists. If Redis cannot be reached, it returns an understandable error.

Your team's second piece: the demonstration app

You build an app inside RocketRide that uses your component. RocketRide suggests Answer Cache, the app on page 2. Their **App Builder** is the feature in the VS Code extension used to create and preview the app.

A sentence you can use in the meeting:
"We are building a component that lets RocketRide save and retrieve information using Redis, plus an app that shows the component working."

The full engagement also includes tests, documentation, a live demo, a blog, and community posts. The client brief describes who helps publish and record those materials. [1]

ROCKETRIDE EXPLAINED / 04

What should the Redis node do?

This is a first proposal to discuss with RocketRide. The meeting should turn it into an agreed feature list.

Scope means deciding exactly what your team will deliver. These proposed actions support Answer Cache and keep the node useful for other apps too.

Proposed action	Why an app might need it
Save and read an entry	Store an answer and retrieve it when the question comes back.
Delete an entry	Remove an answer that should no longer be used.
Set or check expiration	Give an answer a time limit and find out how long it has left.
Increase a counter	Track a number, such as how many questions a user has asked.
Find related entries in batches	Find saved answers for one topic so an authorized user can clear that topic.

The node also needs connection settings

It needs to know where Redis is running and whether a username or password is required. RocketRide should show you how their platform stores these settings securely.

The brief already sets some boundaries

Limit which entries it can touch. The brief requires a configured prefix, a shared beginning on the labels. For example, all allowed labels could begin with "aic:". The node must stay within its assigned group.

Reading is the default. Saving, deleting, and other changes are allowed only after a setting enables them. This is what "read-only unless writes are enabled" means.

Keep actions limited. The brief excludes commands that clear a whole database, change server settings, or run unrestricted scripts. Finding and deleting entries should happen in controlled amounts. [1]

Must-have versus stretch: must-have features are required to finish the agreed project. Stretch features are extras if time allows. Discuss the basic actions above first. More advanced matching and real-time event features need a separate decision.

ROCKETRIDE EXPLAINED / 05

Which app should we propose?

Bring three ideas to discuss. Your team is choosing one demonstration app, not promising to build three.

App idea	What someone would use it for
1. Answer Cache	Ask questions and reuse saved AI answers. This is the app RocketRide proposes in its brief.
2. Session Follow-up Assistant	Save selected facts from a conversation so a later workflow run can retrieve them. Ask how this differs from RocketRide's existing memory feature.
3. Temporary Result Pickup	Save a workflow result, give someone a lookup code, and let them retrieve it before it expires.

Start the discussion with Answer Cache

The screen could contain a question box, an answer, and a label showing whether the answer was newly generated or retrieved from Redis.

RocketRide's proposed app also includes these features:

Recognize repeated questions: decide which wording differences should still reuse an answer.
Limit questions per user: use a counter so one person cannot ask unlimited questions in a minute.
Clear a topic: let an authorized administrator remove outdated answers in one category.
Show results: display how often saved answers were reused and estimate the model cost avoided. [1]

One question that really matters

Should "What is Redis?" and "Explain Redis to me" share an answer? Changing capitalization is simpler than recognizing equivalent meaning. The brief mentions both, so ask them to give examples of what the first version must recognize.

Suggested position:
"Answer Cache looks like a good demonstration. Can we confirm which of its features are required first, especially recognizing differently worded questions?"

The app decides when to reuse an answer. The Redis node saves and retrieves it.

ROCKETRIDE EXPLAINED / 06

How the team gets it done

Personal role assignments remain blank. Use the role descriptions to divide the work together.

Role	What that person owns	Name
Node Lead	Design and build the Redis component; handle code review feedback.	_____
Integration & Test Lead	Help everyone run the project and check that its parts work together.	_____
App Lead	Build the demonstration app and run the live demo.	_____
Content Lead	Coordinate written explanations, posts, meeting logistics, and demo materials.	_____

The dates in RocketRide's proposal

Date	What should be ready
October 30	An agreed design and testing plan.
November 6	Node, tests, and instructions submitted for review as a pull request (PR).
November 13	Review fixes complete and code accepted (merged) into their repository.
November 20	Node released for others to use; app built and tested.
November 25	Live demo and recording. Blog and community posts follow the release.

The brief also schedules social posts for early December. It expects weekly RocketRide engineering calls and separate AIC Board check-ins. Confirm the dates and owners on the call. [1]

Why release timing matters: the brief says the hosted app must use a released node. If release is delayed, its proposed fallback is to demonstrate the app in App Builder preview and wait to publish the release-linked posts.

ROCKETRIDE EXPLAINED / 07

What to say in the meeting

You can ask for a walkthrough. You do not need to arrive knowing every part of RocketRide's code.

Open with this

> "We understand that we are building a component that lets RocketRide save and retrieve information using Redis. Then we will build an app showing it working, potentially by saving and reusing AI answers. Could you show us an existing workflow and where our component should fit?"

Then ask these questions

1. What must our first version do?
Show the proposed actions on page 4. Ask which are required and which can wait.

2. Are we building Answer Cache?
Ask which app features on page 5 are essential. Bring the other two ideas as alternatives.

3. What should count as the same question?
Ask for examples of differently worded questions that should reuse an answer, and examples that should not.

4. Can you show us a working example node?
Ask how to run it, how it receives requests, and how it returns results. The Notion node is the reference suggested in Discord.

5. How do we prove our project is finished?
Ask what the final demo must show and which tests they expect.

6. Who helps with setup, review, and release?
Confirm the right contact, who approves the design, and whether the proposed dates still work.

Read back what you agreed

"We will build _____. Our app will be _____. The required first-version features are _____. We will leave _____ as stretch. Our next deadline is _____, and _____ will review our design."

The PDF is their starting proposal. The later Discord message asks you to bring a rough scope and app ideas that you can refine together. Record the decisions rather than assuming every proposed detail is already settled. [1, 2]

ROCKETRIDE EXPLAINED / 08

How to use the files in VS Code

Read the guide first. Then inspect one source file at a time with a specific question in mind.

Open file	The question it helps answer
Original client PDF	What did RocketRide propose that we deliver?
services.json	How does a node describe itself and the settings a user can choose?
IGlobal.py	How does the node read its settings and prepare to run?
IInstance.py	What actions can this node perform, and what does each return?
test_notion.py	How do developers automatically check whether the existing Notion node behaves correctly?
app-builder.md	How do we create, preview, and publish the demonstration app?

What has been checked so far

During preparation, 78 automated Notion behavior checks and 2 checks of its node description passed. A test is a program that checks an expected result. These results help us understand the example; they do not prove a new Redis node works.

The actual Redis node has not been built. Running Redis on this computer and trying the complete app are still pending. Docker, a tool used to run Redis locally in a packaged environment, was not available. The VS Code workspace is currently open for source browsing.

Your earlier **rocket-ride** folder holds the separate GitHub issue-connector exercise. The **rocketride-server** folder is the actual RocketRide source used for this review. [3]

Where the information comes from

[1] Your supplied RocketRide project PDF, version 2, pages 1-7.
[2] Your Discord screenshot, showing the October 27 preparation request.

[3] Inspected RocketRide source; Notion example; node overview; App Builder guide.

This is your preparation guide, rewritten in plain language. RocketRide's original brief is unchanged. Names remain blank so your team can choose ownership together.